

THỰC HÀNH BUỔI 3

TÌM KIẾM (BFS, UCS, DFS)

1. Node

- Định nghĩa một nút
- Có thuộc tính là name (kiểu chuỗi)
- Phương thức định nghĩa: `get_name`. Trả về tên của nút
- Định nghĩa lại: `__str__`

```
class Node:
    def __init__(self, n):
        self.name = n
    def get_name(self):
        return self.name
    def __str__(self):
        return self.name
```

2. Edge:

- Định nghĩa một cạnh
- Thuộc tính: nút nguồn (src), nút đích (dest), chi phí đi từ nút nguồn sang nút đích (cost)
- Các phương thức: lấy nút nguồn (`get_source`), lấy nút đích (`get_dest`), lấy chi phí (`get_cost`)
- Phương thức định nghĩa lại: `__str__`

```
class Edge:
    def __init__(self, s, d, c):
        self.src = s
        self.dest = d
        self.cost = c
    def get_source(self):
        return self.src
    def get_dest(self):
        return self.dest
    def get_cost(self):
        return self.cost
    def __str__(self):
        return '{}->{}'.format(self.src, self.dest)
```

3. Graph:

- Định nghĩa một đồ thị có hướng
- Thuộc tính edges: danh sách các cạnh được định nghĩa là dictionary với khóa là nút nguồn và giá trị là dictionary nút đích

Ví dụ: Cạnh AB (chi phí 2), AC (chi phí là 4)

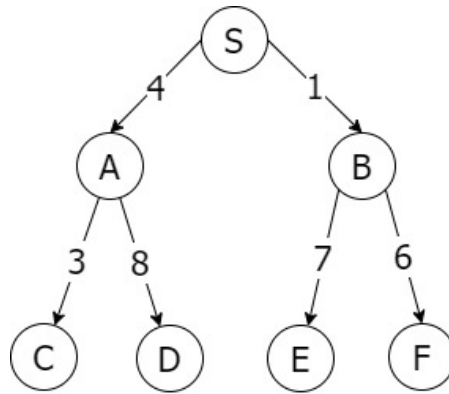
{A: {B: 2, C: 4}}

– Lớp được cài đặt

```
class Graph:
    def __init__(self):
        self.edges = {} # các cạnh
    # thêm một nút
    def add_node(self, n):
        # nếu nút đã có trong danh sách các cạnh => báo lỗi
        if n in self.edges:
            print('Nút đã tồn tại')
        # nếu nút chưa có thì thêm phần tử vào dictionary
        # với khóa là nút và giá trị là một danh sách rỗng
        else:
            self.edges[n] = {}
    # thêm một cạnh
    def add_edge(self, e:Edge):
        src = e.get_source()
        dest = e.get_dest()
        cost = e.get_cost()
        # nếu nút nguồn hoặc nút đích không có trong danh sách thì báo lỗi
        if (src not in self.edges) or (dest not in self.edges):
            print('Nút không có trong đồ thị')
        else:
            self.edges[src][dest] = cost
    # lấy một nút con
    def get_children(self, n):
        return self.edges[n]
    # kiểm tra một nút tồn tại không
    def has_node(self, n):
        return n in self.edges
    # lấy một nút dựa trên nhãn
    def get_node(self, name):
        for n in self.edges:
            if n.get_name() == name:
                return n
        return None
    def get_cost(self, src, dest):
        return self.edges[src][dest]
    # trả về chuỗi mô tả đồ thị
    def __str__(self):
        result = ''
        for src in self.edges:
            for dest in self.edges[src]:
                result = result + '{}->{}'.format(src,dest) + '\n'
        return result[:-1]
```

4. Phương thức xây dựng đồ thị:

Giả sử cho đồ thị như



```

def build_graph():
    A = Node('A'); B = Node('B'); C = Node('C'); D = Node('D'); E = Node('E')
    F = Node('F'); S = Node('S')

    edges = []
    edges.append(Edge(S, A, 4)); edges.append(Edge(S, B, 1))
    edges.append(Edge(A, C, 3)); edges.append(Edge(A, D, 8))
    edges.append(Edge(B, E, 7)); edges.append(Edge(B, F, 6))

    my_graph = Graph()
    my_graph.add_node(A); my_graph.add_node(B)
    my_graph.add_node(C); my_graph.add_node(D)
    my_graph.add_node(E); my_graph.add_node(F)
    my_graph.add_node(S)

    for e in edges:
        my_graph.add_edge(e)
    return my_graph
  
```

5. Phương thức print_path

– In đường đi tìm được

– Input: {nút_con: nút cha,...}

Nếu nút cha là S thì nút cha là None

– In ra danh sách đường đi tìm được dựa vào input

```

def print_path(path, goal):
    res = ''
    n = goal
    while n != None:
        res = str(n) + '=>' + res
        n = path[n]
    print(res[:-2])
  
```

6. Phương thức get_cost_path

– Tìm chi phí đường đi

– Input: {nút_con: nút cha,...}

– Kết quả trả ra là chi phí ứng với đường đi

```
def get_cost_path(graph, path, goal):
    cost = 0
    n = goal
    while n != None:
        if (path[n] != None):
            cost += graph.get_cost(path[n], n)
            n = path[n]
    return cost
```

7. Phương thức bfs

```
def bfs(graph: Graph, start, goal):
    frontier = [start] # tập đường biên
    path = {start: None} # lưu đường đi dựa vào quan hệ nút cha và nút con
    explored = [] # tập các nút đã duyệt
    found = False
    print('{:<20s} | {:<20s}'.format('Tập biên', 'Tập đã duyệt'))
    print('-' * 32)
    print('{:<20s} | {:<20s}'.format(str(start), ''))
    while len(frontier) > 0:
        node = frontier.pop(0)
        if node == goal: # nếu là nút đích thoát khỏi vòng lặp while
            found = True
            break
        explored.append(node) # nếu không là nút đích, thêm vào tập duyệt
        for n in graph.get_children(node): # xét các nút con và thêm vào tập biên
            if n not in frontier and n not in explored:
                frontier.append(n)
                path[n] = node
        # in ra quá trình duyệt
        str1 = ''
        for n in frontier:
            str1 += str(n) + ','
        str1 = str1[:-1]
        str2 = ''
        for n in explored:
            str2 += str(n) + ','
        str2 = str2[:-1]
        print('{:<20s} | {:<20s}'.format(str1, str2))

    if found: return path
    else: return None
```

8. Phương thức dfs:

```

def dfs(graph: Graph, start, goal):
    frontier = [start]
    path = {start: None}
    explored = []
    found = False

    print('{:<20s} | {:<20s}'.format('Tập biên', 'Tập đã duyệt'))
    print('-' * 32)
    print('{:<20s} | {:<20s}'.format(str(start), ''))

    while len(frontier) > 0:
        node = frontier.pop(0)
        if node == goal:
            found = True; break

        explored.append(node)
        temp = []
        for n in graph.get_children(node):
            if n not in frontier and n not in explored:
                temp.append(n)
                path[n] = node
        frontier = temp + frontier

        str1 = ''
        for n in frontier:
            str1 += str(n) + ','
        str1 = str1[:-1]
        str2 = ''
        for n in explored:
            str2 += str(n) + ','
        str2 = str2[:-1]
        print('{:<20s} | {:<20s}'.format(str1, str2))

    if found: return path
    else: return None

```

9. Thực hiện

```
graph = build_graph() # tạo đồ thị
start = graph.get_node('S') # xác định nút gốc
goal = graph.get_node('F') # xác định nút đích

print('\n Tìm kiếm theo chiều rộng \n')
path = bfs(graph, start, goal) # tìm kiếm theo chiều rộng
if path != None: # nếu tìm được thì in ra kết quả
    print()
    print('Đường đi từ {} đến {}: '.format(str(start),
                                             str(goal)), end="")

    print_path(path, goal)
    print('Chi phí đường đi: {}'.format(get_cost_path(graph, path, goal)))
else:
    print('Không tìm thấy đường đi')

print('\n Tìm kiếm theo chiều sâu \n')
path = dfs(graph, start, goal) # tìm kiếm theo chiều sâu
if path != None:
    print()
    print('Đường đi từ {} đến {}: '.format(str(start),
                                             str(goal)), end="")

    print_path(path, goal)
    print('Chi phí đường đi: {}'.format(get_cost_path(graph, path, goal)))
else:
    print('Không tìm thấy đường đi')
```